Exercices — Chapitre 7 : Adresses mémoire, pointeurs et références

Cours associé : `C07_Adresses_pointeurs_references` · **Fichiers de départ :** `02_Exercices/fichiers/E07/`

> **Méthode :** pour chaque pointeur, dessinez un schéma « boîtes et flèches » comme celui du cours (§1) : une boîte par variable, une flèche de chaque pointeur vers la variable qu'il désigne. Mettez le schéma à jour à chaque instruction qui modifie un pointeur.

N°	Titre	Niveau	Notions du cours
7.1	Adresse et modification	1 — Application	§1
7.2	Prévoir l'affichage	1 — Application	§1, §5
7.3	Fonction et pointeur	2 — Entraînement	§1.1, §2
7.4	Parcours d'un tableau	2 — Entraînement	§3
7.5	Choisir le passage des paramètres	3 — Problème	§5.2, ch. 5 §6
7.6	Rechercher un capteur	4 — Défi	§2, §4, §5

Niveau 1 — Application directe

Exercice 7.1 — Adresse et modification

Objectif : déclarer un pointeur, afficher une adresse, modifier une variable à travers un pointeur.

Énoncé :

1. Déclarez un entier initialisé à 25 et un pointeur vers cet entier.
2. Affichez la valeur de l'entier, son adresse (`&`), le contenu du pointeur et la valeur pointée (`*`).
3. Modifiez la valeur **à travers le pointeur**, puis affichez la variable d'origine.

Résultat attendu : l'adresse et le contenu du pointeur sont identiques. Ils s'affichent en hexadécimal, par exemple `0x7ffd5c1e8a4c`, et changent d'une exécution à l'autre. La variable d'origine affiche la nouvelle valeur.

Exercice 7.2 — Prévoir l'affichage

Objectif : suivre un pointeur qui change de cible et une référence.

Énoncé : sans exécuter le programme `ex7_2.cpp`, indiquez ce qu'il affiche. Suivez, ligne par ligne, la variable désignée par `p`. Vérifiez ensuite en compilant.

```cpp
#include <iostream>

int main() {
    int a{4};
    int b{9};
    int* p{&a};
    int& r{b};

    *p = *p + 1;
    p = &b;
    *p = *p * 2;
    r = r - 3;

    std::cout << a << ' ' << b << '\n';

    return 0;
}
```

Résultat attendu : un tableau donnant, après chaque instruction, la cible de `p`, la valeur de `a` et la valeur de `b`, puis la ligne affichée.

Niveau 2 — Entraînement

Exercice 7.3 — Fonction et pointeur

Objectif : écrire une fonction qui reçoit un pointeur éventuellement nul.

Énoncé :

1. Écrivez `void doublerSiPossible(int* ptr)`, qui double la valeur pointée si le pointeur n'est pas nul et ne fait rien sinon.
2. Testez-la avec l'adresse d'une variable valant 21, puis avec `nullptr`.

Résultat attendu :

```
Apres appel avec une adresse : 42
Apres appel avec nullptr : 42
```

Exercice 7.4 — Parcours d'un tableau

Objectif : parcourir un tableau avec un pointeur et comparer avec les indices.

Énoncé : soit le tableau `int valeurs[5]{3, 8, 1, 9, 4};`.

1. Affichez ses éléments en utilisant un pointeur et `*(ptr + i)`.
2. Affichez-les avec des indices (`valeurs[i]`), puis avec une boucle sur un intervalle.
3. Laquelle des trois écritures est la plus lisible ? Laquelle permet le plus facilement une erreur de dépassement ?

Résultat attendu : trois fois la ligne `3 8 1 9 4`, et une réponse argumentée à la question 3.

Niveau 3 — Résolution de problèmes

Exercice 7.5 — Choisir le passage des paramètres

Objectif : choisir, pour chaque besoin, entre valeur, référence, référence constante et pointeur.

Énoncé :

1. Pour chaque fonction décrite, écrivez le **prototype** le plus adapté et justifiez votre choix en une phrase.

Fonction	Description
`aireRectangle`	Calcule l'aire d'un rectangle à partir de sa longueur et de sa largeur
`remettreAZero`	Remet à zéro un compteur entier de l'appelant
`afficherRapport`	Affiche un rapport à partir d'un `std::vector<double>` de 10 000 mesures
`afficherNomEquipement`	Affiche le nom d'un équipement, fourni sous forme de `std::string`
`mettreAJourCapteur`	Remplace la valeur d'une `Mesure` (ch. 5 §7) **si elle existe** ; l'appelant peut ne pas en avoir

2. Écrivez et testez `remettreAZero` et `mettreAJourCapteur` (cette dernière reçoit aussi la nouvelle valeur).

Contraintes : aucune copie inutile d'un objet volumineux ; aucun pointeur là où une référence suffit.

Niveau 4 — Défi

Exercice 7.6 — Rechercher un capteur

Objectif : renvoyer un pointeur pour exprimer « trouvé ou absent », puis agir à travers ce pointeur.

Contexte : le logiciel de supervision conserve le parc de capteurs dans un `std::vector<Mesure>`, avec la `struct Mesure` du chapitre 5 (`capteur`, `valeur`). Une recherche par identifiant peut échouer : le pointeur nul représente ce cas.

Énoncé :

1. Écrivez `Mesure* chercherCapteur(std::vector<Mesure>& parc, const std::string& identifiant)`. Elle renvoie l'adresse de la mesure dont le champ `capteur` est égal à `identifiant`, ou `nullptr` si aucune ne correspond.
2. Dans `main()`, créez le parc `{"T1", 21.5}`, `{"T2", 22.0}`, `{"H1", 45.0}`.
3. Recherchez `T2` : s'il est trouvé, remplacez sa valeur par `23.4` **à travers le pointeur** (`->`).
4. Recherchez `X9` et affichez un message s'il est absent.
5. Affichez enfin tout le parc.

Contraintes :

- testez toujours le pointeur avant de l'utiliser ;

- n'ajoutez aucun élément au vecteur entre la recherche et l'utilisation du pointeur : en agrandissant le vecteur, `push_back` peut déplacer ses éléments en mémoire, et le pointeur désignerait alors une zone qui n'est plus valide.

Résultat attendu :

```
T2 mis a jour : 23.4
X9 introuvable
T1 : 21.5
T2 : 23.4
H1 : 45
```